



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

Detección del parásito Varroa
Destructor mediante redes
neuronales



Presentado por Manuel Meléndez Ortega
en Universidad de Burgos — 10 de junio
de 2026

Tutor: Carlos Cambra Baseca

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de proyecto software	1
A.1. Introducción	1
A.2. Planificación temporal y metodología	1
A.3. Estudio de viabilidad	6
Apéndice B Especificación de requisitos	9
B.1. Introducción	9
B.2. Objetivos generales	9
B.3. Catálogo de requisitos	9
B.4. Especificación de requisitos	12
Apéndice C Especificación de diseño	19
C.1. Introducción	19
C.2. Diseño de datos	19
C.3. Diseño arquitectónico	24
C.4. Diseño procedimental	27
Apéndice D Documentación técnica de programación	29
D.1. Introducción	29
D.2. Estructura de directorios	29
D.3. Manual del programador	31

D.4. Compilación, instalación y ejecución del proyecto	32
D.5. Pruebas del sistema	34
Apéndice E Documentación de usuario	37
E.1. Introducción	37
E.2. Requisitos de usuarios	37
E.3. Instalación	38
E.4. Manual del usuario	39
Apéndice F Anexo de sostenibilización curricular	45
F.1. Introducción	45
F.2. Conocimiento y conciencia ambiental	45
F.3. Innovación y tecnología para la sostenibilidad	46
F.4. Despliegue local y accesibilidad tecnológica	46
F.5. Responsabilidad social y ética profesional	46
F.6. Aplicación de competencias	46
F.7. Reflexión personal	47
Bibliografía	49

Índice de figuras

A.1. Cantidad de horas estimadas frente a las reales.	3
A.2. Porcentaje de horas extra frente a las estimadas.	4
A.3. Porcentaje aproximado de tiempo para cada fase.	5
A.4. Gráfico Gantt con la evolución temporal de las tareas.	5
B.1. Diagrama de casos de uso general.	12
C.1. Ejemplo de la evolución de las métricas de error durante el entrenamiento.	22
C.2. Ejemplo de detección del ácaro tras el entrenamiento del modelo.	23
C.3. Avance de la plaga a lo largo del tiempo.	25
C.4. Diagrama de secuencia de comunicación cliente-servidor para el proceso de análisis.	28
D.1. Consola de comandos para Spring Boot lista.	33
D.2. Consola de comandos para el microservicio lista.	33
D.3. Línea de comandos con el entorno activo.	34
E.1. Pantalla de login.	39
E.2. Pantalla de registro.	39
E.3. Pantalla principal.	40
E.4. Pantalla para nuevos análisis.	41
E.5. Parte superior de la pantalla del historial.	42
E.6. Parte inferior de la pantalla del historial.	42
E.7. Opción para visualizar las imágenes con las detecciones de la IA.	43
E.8. Botón para acceder al panel admin.	43
E.9. Pantalla de administración.	44

Índice de tablas

A.1. Presupuesto estimado del proyecto simulando un entorno real. .	6
B.1. Especificación del caso de uso CU-1.	13
B.2. Especificación del caso de uso CU-2.	14
B.3. Especificación del caso de uso CU-3.	15
B.4. Especificación del caso de uso CU-4.	16
B.5. Especificación del caso de uso CU-5.	17
B.6. Especificación del caso de uso CU-6.	18
D.1. Herramientas y versiones de software utilizadas.	32

Apéndice A

Plan de proyecto software

A.1. Introducción

Cualquier desarrollo de software necesita una buena hoja de ruta para no perder el rumbo, y más aún cuando el proyecto es un híbrido que mezcla la investigación pura en Inteligencia Artificial con el desarrollo de una plataforma web. En este anexo se detalla cómo se ha organizado el tiempo, qué metodologías se han elegido para cada parte y cuánto costaría llevar este sistema de detección de varroa al mundo real.

A.2. Planificación temporal y metodología

Dado que este TFG une la ciencia de datos y la ingeniería de software, se decidió dividir el enfoque en dos:

Para la parte de la Inteligencia Artificial, se utilizó la metodología **CRISP-DM** (Cross-Industry Standard Process for Data Mining) [11]. Es el estándar más utilizado en proyectos de aprendizaje automático porque permite avanzar y retroceder de fase según los descubrimientos que se hagan con los datos.

Por otro lado, para la parte del **desarrollo web**, se optó por una adaptación ágil basada en **Scrum** y **Kanban**, dividiendo el trabajo en ciclos cortos y usando tableros visuales para programar el servidor y la interfaz.

Uniando ambas filosofías, el trabajo total se organizó en los siguientes bloques:

1. **Arranque y viabilidad:** Búsqueda de información sobre la varroa, decidir el stack tecnológico y plantear los objetivos del TFG.
2. **Preparación de los datos (dataset):** Recopilación y análisis de las 429 imágenes originales. En esta fase se aplicaron técnicas de aumento de datos (*data augmentation*) con la librería Albumentations [1] para solucionar el problema de la escasez de muestras.
3. **Entrenamiento del modelo:** Configuración de la red neuronal Faster R-CNN [7] con ResNet-50 [3] utilizando el entorno PyTorch [6]. Se aplicó aprendizaje por transferencia (*Transfer Learning*) con el dataset COCO [5] para acelerar el proceso.
4. **Desarrollo del servidor (backend):** Programación en Java con el framework Spring Boot [10] como el núcleo de la aplicación.
5. **Interfaz Web (frontend):** Diseño de las pantallas utilizando el motor de plantillas Thymeleaf, HTML5 y CSS.
6. **Despliegue y pruebas:** Implementación de un script de automatización (`iniciar_todo.bat`) capaz de configurar el entorno desde cero. Este ejecutable se encarga de aislar todo el entorno del sistema de forma nativa, creando el entorno virtual (`.venv`) de Python con sus dependencias correspondientes, inicializando la base de datos local embebida y arrancando simultáneamente el servidor web y la API de Inteligencia Artificial. Finalmente, se llevaron a cabo las pruebas de integración de todo el ecosistema.
7. **Documentación:** Redacción final de la memoria del proyecto y sus anexos.

En un principio, se estimó que todo esto llevaría unas 300 horas. Sin embargo, los problemas derivados de la baja resolución de las imágenes y la necesidad de ajustar los hiperparámetros del modelo hicieron que el tiempo total empleado rondara las 315 horas.

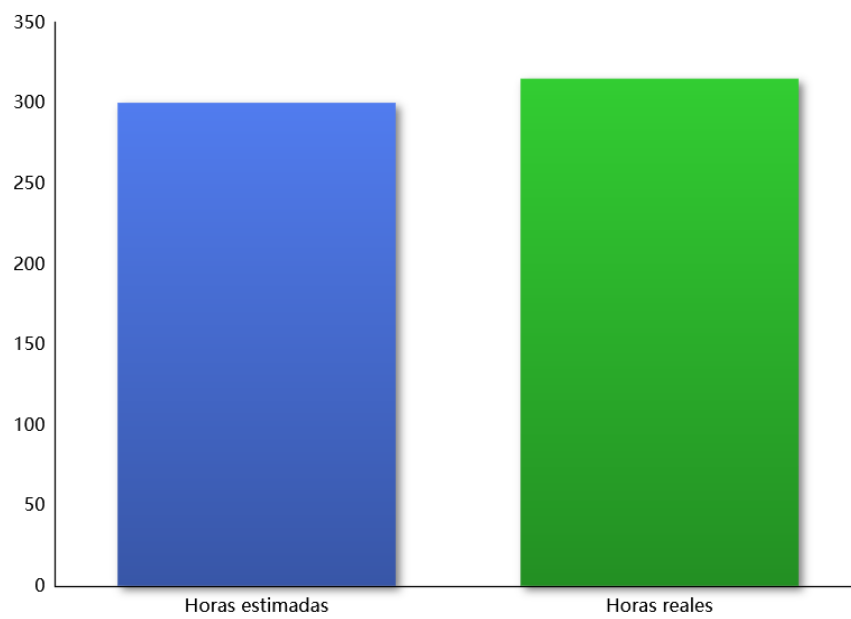


Figura A.1: Cantidad de horas estimadas frente a las reales.

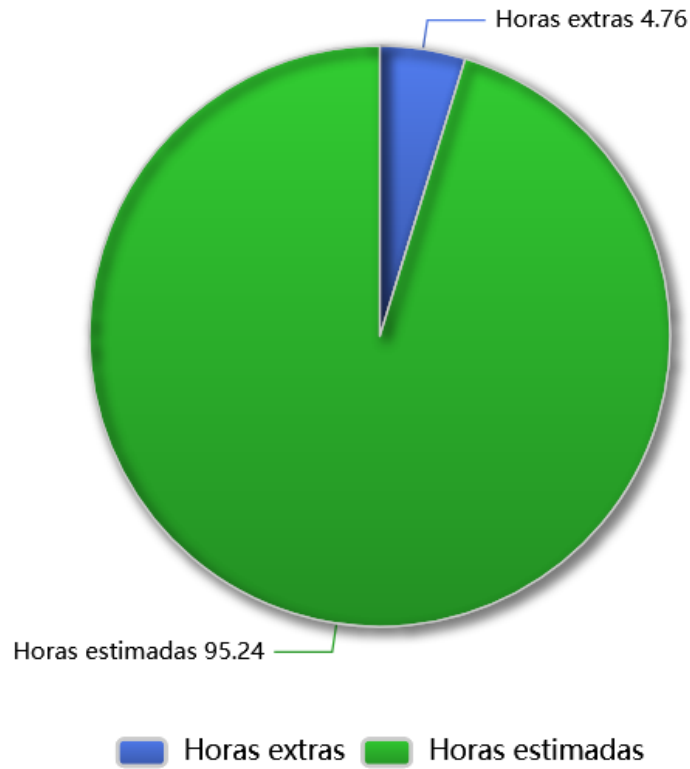


Figura A.2: Porcentaje de horas extra frente a las estimadas.

1

Si repartimos esas horas entre las grandes fases del proyecto, los porcentajes quedan de la siguiente manera:

- **10 % Tareas previas:** Planificación, recopilación y aumento de datos del dataset.
- **35 % Desarrollo de la IA:** Entrenamiento y validación del modelo en PyTorch.
- **20 % Desarrollo web y despliegue:** Creación de la plataforma en Spring Boot y su despliegue local.
- **35 % Documentación:** Redacción del TFG.

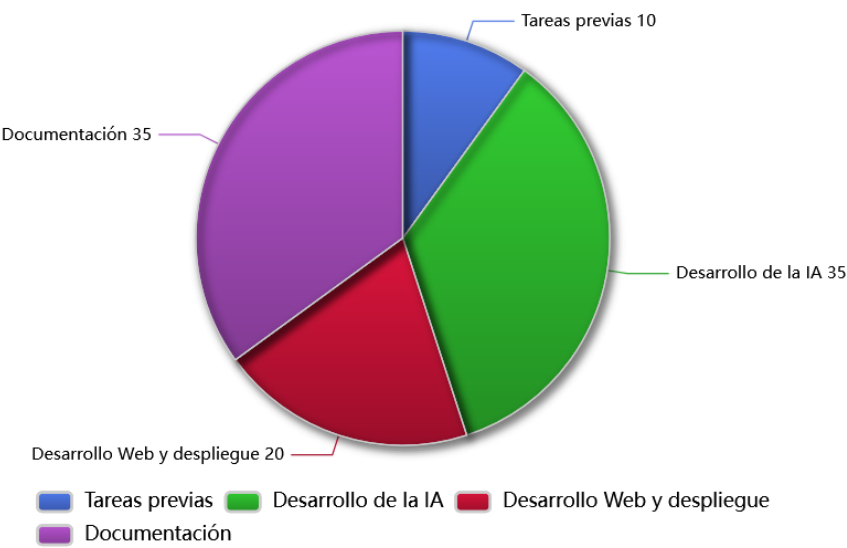


Figura A.3: Porcentaje aproximado de tiempo para cada fase.

1

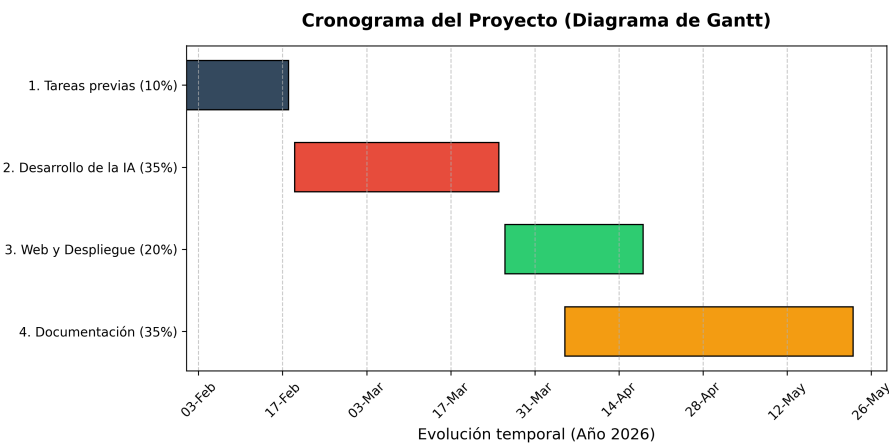


Figura A.4: Gráfico Gantt con la evolución temporal de las tareas.

1

A.3. Estudio de viabilidad

Viabilidad económica

Para saber si este proyecto tendría sentido en el mercado real, vamos a suponer que se desarrolla en un entorno empresarial, clasificando los gastos en fijos, variables, directos e indirectos.

Costes fijos:

- Alquiler de un pequeño local u oficina.
- El sueldo de un desarrollador de software junior.

Costes variables:

- Facturas de electricidad y conexión a internet.

Costes directos:

- Un ordenador de alto rendimiento equipado con una tarjeta gráfica NVIDIA, indispensable para procesar en paralelo el entrenamiento del modelo.
- Una cámara de fotos decente para ir a los colmenares a sacar muestras.

Costes indirectos:

- Amortización y mantenimiento del ordenador y equipo informático.

Concepto	Gasto estimado
Alquiler de la oficina	800,00€
Sueldo del desarrollador (aprox. 315h)	4.725,00€
Luz e internet	210,00€
Cámara fotográfica	350,00€
Ordenador de desarrollo	1.400,00€
Mantenimiento informático	80,00€
COSTE TOTAL	7.565,00€

Tabla A.1: Presupuesto estimado del proyecto simulando un entorno real.

Posibles vías de negocio

Si esta herramienta saliera al mercado, podría ofrecerse a los apicultores de tres formas distintas para recuperar la inversión:

- **Plan básico (gratis):** Pensado para apicultores aficionados con pocas colmenas. Permitiría analizar un número limitado de imágenes al mes.
- **Plan profesional:** Enfocado a apicultores profesionales. Mediante una suscripción, tendrían análisis ilimitados y almacenamiento de su histórico de infestación en la base de datos.
- **Plan empresas:** Dirigido a cooperativas agrarias, ofreciéndoles despliegues en servidores dedicados y la posibilidad de centralizar todos los diagnósticos de sus socios.

Viabilidad legal y licencias

Todo el ecosistema tecnológico utilizado en este proyecto es de código abierto, por lo que el coste en licencias de software es de 0€.

Para mantener esta filosofía, se ha decidido que el código fuente de esta aplicación se publique bajo la **licencia MIT**. Se ha elegido esta licencia por ser una de las más sencillas y permisivas: permite que cualquier persona pueda utilizar el código, modificarlo e incluso comercializarlo, con la única condición legal de incluir un aviso reconociendo la autoría original del software.

Apéndice B

Especificación de requisitos

B.1. Introducción

Este apéndice detalla la especificación de requisitos del sistema desarrollado. Se definen los objetivos generales de la aplicación, se categorizan los requisitos en funcionales y no funcionales, y finalmente se detallan los casos de uso principales para documentar la interacción entre el usuario y el sistema.

B.2. Objetivos generales

El objetivo principal del sistema es automatizar la detección y el conteo del ácaro *Varroa Destructor* en imágenes de bandejas sanitarias mediante técnicas de visión artificial. Para lograrlo, la aplicación proporciona una interfaz web intuitiva en entorno local que permite a los apicultores cargar fotografías, procesarlas mediante un modelo de aprendizaje profundo y obtener un recuento visual y numérico preciso que facilite la toma de decisiones sanitarias en la colmena.

B.3. Catálogo de requisitos

Requisitos funcionales

Los requisitos funcionales describen las acciones específicas y los servicios que el sistema debe ser capaz de ejecutar:

- **RF-1: Gestión de usuarios y accesos:** El sistema debe restringir las funcionalidades principales a usuarios registrados y autenticados.
 - **RF-1.1: Registro y login:** Permitir a los apicultores crear una cuenta mediante un nombre de usuario único y autenticarse de forma segura en la plataforma mediante contraseña.
 - **RF-1.2: Control de roles:** Diferenciar entre el rol de usuario estándar y el rol de administrador del sistema.
- **RF-2: Carga y gestión de muestreos:** El usuario debe ser capaz de gestionar las imágenes de sus colmenas dentro del entorno local.
 - **RF-2.1: Carga múltiple:** Permitir al usuario seleccionar y cargar una o múltiples imágenes simultáneamente desde su almacenamiento local.
 - **RF-2.2: Saneamiento de archivos:** El sistema debe renombrar las imágenes subidas utilizando identificadores únicos (UUID) para evitar colisiones de nombres en el servidor local.
- **RF-3: Procesamiento de imágenes:** El sistema procesará de forma automatizada las imágenes para localizar la plaga.
 - **RF-3.1: Delegación REST:** El backend en Java Spring Boot debe redirigir la ruta de la imagen local de forma transparente hacia el microservicio de Python Flask.
 - **RF-3.2: Detección de objetos:** Ejecutar el modelo de red neuronal Faster R-CNN [7] para localizar con precisión las instancias del ácaro *Varroa Destructor*.
- **RF-4: Visualización e interpretación de resultados:** Mostrar la información de los análisis de manera clara para la toma de decisiones del apicultor.
 - **RF-4.1: Renderizado de cajas delimitadoras:** Generar una representación visual donde se resalten los parásitos detectados mediante *bounding boxes*.
 - **RF-4.2: Conteo de parásitos:** Calcular y mostrar en pantalla el número total de varroas detectadas tanto por imagen individual como por el lote completo de carga.
 - **RF-4.3: Historial y análisis evolutivo:** Mantener un registro histórico por usuario y calcular automáticamente si el nivel de infestación sube o baja comparándolo con el ciclo biológico previo de 21 días [9].

- **RF-5: Mantenimiento del sistema (panel de administración):** Proporcionar herramientas de control interno para el estado del servidor local.
 - **RF-5.1: Saneamiento del servidor:** Permitir al administrador forzar una limpieza del sistema de archivos, eliminando de forma automática aquellas imágenes huérfanas que no estén enlazadas en la base de datos.
 - **RF-5.2: Gestión e integridad de cuentas:** Permitir de manera exclusiva al rol de administrador dar de baja o eliminar cuentas de usuarios locales de la base de datos para controlar el acceso al servidor.

Requisitos no funcionales

Los requisitos no funcionales definen los criterios de calidad, restricciones técnicas y propiedades de seguridad que debe cumplir el sistema desarrollado, garantizando su correcto funcionamiento en el entorno de ejecución local:

- **RNF-1: Usabilidad y curva de aprendizaje:** La interfaz web de la aplicación debe ser limpia, intuitiva y autodescriptiva. Está diseñada utilizando principios de diseño responsivo para que un apicultor pueda visualizar los resultados cómodamente desde su dispositivo sin interactuar con terminales de comandos.
- **RNF-2: Eficiencia y tiempo de respuesta:** El proceso de comunicación interna mediante la API REST local entre el backend de Java y el microservicio de Inteligencia Artificial debe estar optimizado para garantizar una experiencia de usuario fluida en el navegador.
- **RNF-3: Restricciones de hardware y portabilidad:** Dado que la aplicación se ejecuta de manera estrictamente local, el sistema debe ser capaz de arrancar e invocar el modelo de visión artificial en hardware convencional.
- **RNF-4: Seguridad y confidencialidad de accesos:** A pesar de ser un entorno local, las contraseñas introducidas en el formulario de registro se almacenan de forma segura en la base de datos aplicando un algoritmo de encriptación y *hashing* seguro como es BCrypt, gestionado a través de Spring Security [10].

- **RNF-5: Integridad y gestión del almacenamiento:** El sistema debe garantizar que el disco duro local no se sature debido al flujo continuo de imágenes analizadas, ofreciendo mecanismos de saneamiento internos que aseguren que cualquier archivo huérfano pueda ser eliminado de manera segura por el administrador.

B.4. Especificación de requisitos

Para ilustrar de forma clara las interacciones entre los diferentes actores del sistema y los procesos principales de la plataforma local, se modela el diagrama de casos de uso general.

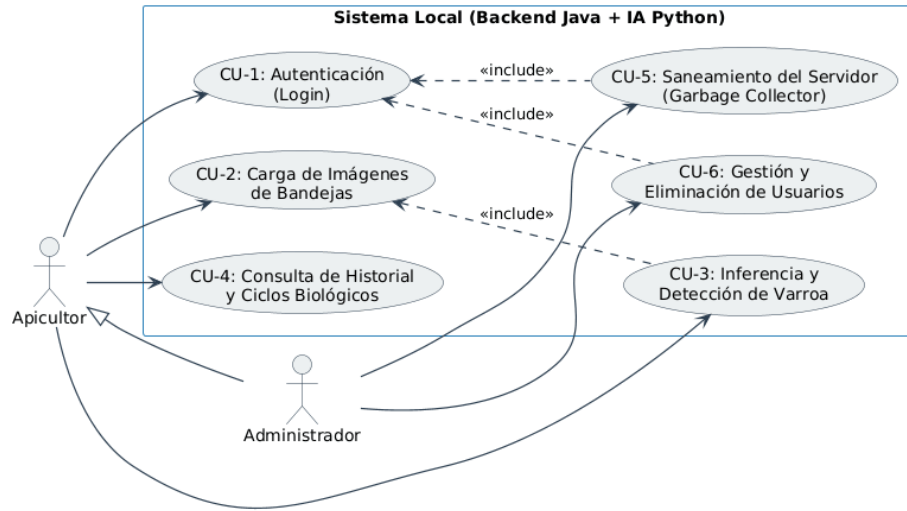


Figura B.1: Diagrama de casos de uso general.

1 A continuación, se detalla la especificación formal de cada uno de los casos de uso principales identificados en el sistema:

CU-1	Autenticación en el sistema (login)
Versión	1.0
Autor	Manuel Meléndez Ortega
Requisitos asociados	RF-1, RF-1.1
Descripción	El usuario introduce sus credenciales para acceder de forma segura a las herramientas de análisis de la plataforma local.
Precondición	El usuario debe estar registrado en la base de datos H2 [2] local.
Relaciones	Es incluido por CU-5 y CU-6.
Acciones	1. El usuario accede a la pantalla inicial. 2. Introduce su nombre de usuario y contraseña. 3. El backend verifica el <i>hash</i> BCrypt de la contraseña. 4. Se concede el acceso y se redirige al panel principal.
Postcondición	Se establece una sesión activa para el usuario.
Excepciones	Contraseña incorrecta o usuario no encontrado (se muestra mensaje de error).
Importancia	Alta

Tabla B.1: Especificación del caso de uso CU-1.

CU-2	Carga de imágenes de bandejas
Versión	1.0
Autor	Manuel Meléndez Ortega
Requisitos asociados	RF-2, RF-2.1, RF-2.2
Descripción	El apicultor selecciona las fotografías tomadas para subirlas al servidor local para su procesamiento.
Precondición	El usuario ha iniciado sesión correctamente (CU-1).
Relaciones	Es incluido por CU-3.
Acciones	1. El usuario pulsa el botón de carga de archivos. 2. Selecciona una o múltiples imágenes desde su dispositivo local. 3. El sistema valida que el formato de archivo sea correcto (JPG/PNG). 4. El backend de Java renombra los archivos utilizando identificadores únicos (UUID). 5. Los archivos se guardan físicamente en el directorio local de subidas.
Postcondición	Las imágenes quedan almacenadas y listas para la inferencia.
Excepciones	Intento de subida de archivos con formato no soportado.
Importancia	Alta

Tabla B.2: Especificación del caso de uso CU-2.

CU-3	Inferencia y detección de varroa
Versión	1.0
Autor	Manuel Meléndez Ortega
Requisitos asociados	RF-3, RF-3.1, RF-3.2, RF-4.1, RF-4.2
Descripción	El sistema procesa las imágenes para localizar los ácaros, delegando la carga computacional en el microservicio de Python.
Precondición	Se ha completado la carga de imágenes exitosamente (CU-2).
Relaciones	«include» a CU-2.
Acciones	1. Java realiza una petición REST a Flask enviando la ruta de la imagen. 2. Flask procesa la imagen usando el modelo Faster R-CNN [7]. 3. Se dibujan las cajas delimitadoras sobre los parásitos. 4. Flask devuelve el conteo total a Java. 5. Java renderiza la vista con la imagen procesada y el total de detecciones.
Postcondición	El análisis queda asociado a la sesión del usuario en la base de datos.
Excepciones	Fallo de conexión entre el servidor Java y el microservicio Flask.
Importancia	Alta

Tabla B.3: Especificación del caso de uso CU-3.

CU-4	Consulta de historial y ciclos biológicos
Versión	1.0
Autor	Manuel Meléndez Ortega
Requisitos asociados	RF-4.3
Descripción	El apicultor revisa su historial de muestreos para comprobar la evolución de la plaga.
Precondición	El usuario tiene análisis previos guardados en su cuenta.
Relaciones	Ninguna.
Acciones	1. El usuario accede a la sección de "Historial". 2. El sistema recupera de la base de datos los conteos anteriores. 3. El sistema compara automáticamente el conteo actual con el de hace 21 días (ciclo de eclosión) [9]. 4. Se muestra un indicador de si el tratamiento ha sido efectivo o la plaga ha aumentado.
Postcondición	El usuario obtiene información valiosa para la toma de decisiones.
Excepciones	No hay análisis registrados hace 21 días (se muestra solo el conteo actual sin comparativa).
Importancia	Media

Tabla B.4: Especificación del caso de uso CU-4.

CU-5	Saneamiento del servidor (garbage collector)
Versión	1.0
Autor	Manuel Meléndez Ortega
Requisitos asociados	RF-5, RF-5.1
Descripción	El administrador del sistema realiza un barrido para borrar imágenes antiguas y evitar la saturación del disco local.
Precondición	El usuario autenticado debe tener el rol de administrador.
Relaciones	«include» a CU-1 (Login).
Acciones	1. El administrador accede al panel de control oculto. 2. Ejecuta la rutina de "Limpieza de archivos". 3. El backend verifica físicamente qué imágenes UUID no constan en la base de datos. 4. El sistema elimina físicamente esas imágenes huérfanas. 5. Se muestra un resumen del espacio en disco liberado.
Postcondición	El almacenamiento local queda optimizado de manera segura.
Excepciones	Permisos insuficientes de escritura en la carpeta del sistema.
Importancia	Media

Tabla B.5: Especificación del caso de uso CU-5.

CU-6	Gestión y eliminación de usuarios
Versión	1.0
Autor	Manuel Meléndez Ortega
Requisitos asociados	RF-5, RF-5.2
Descripción	El administrador accede al listado de usuarios locales registrados en el sistema y elimina una cuenta que ya no sea necesaria.
Precondición	El usuario autenticado debe tener el rol de administrador.
Relaciones	«include» a CU-1 (Login).
Acciones	1. El administrador accede a la sección de "Gestión de Usuarios". 2. El sistema recupera y muestra el listado de usuarios desde la base de datos. 3. El administrador selecciona un usuario y pulsa el botón "Eliminar". 4. El sistema solicita una confirmación de la acción. 5. Tras confirmar, el backend ejecuta la baja en la base de datos.
Postcondición	El usuario es eliminado del sistema y ya no podrá iniciar sesión.
Excepciones	Intento de eliminarse a sí mismo (el sistema bloquea la acción para evitar dejar el servidor sin administrador).
Importancia	Media

Tabla B.6: Especificación del caso de uso CU-6.

Apéndice C

Especificación de diseño

C.1. Introducción

Este apéndice detalla la arquitectura técnica y el diseño interno del sistema de detección de *Varroa Destructor*. Se describe cómo se estructuran los datos para el aprendizaje profundo, la organización de los componentes del sistema bajo un patrón cliente-servidor y el flujo lógico que siguen las peticiones desde que el apicultor sube una imagen hasta que se renderizan los resultados.

C.2. Diseño de datos

El diseño de datos en el ámbito del aprendizaje profundo es un pilar fundamental para garantizar el correcto rendimiento del modelo de Inteligencia Artificial. A continuación, se detalla el origen y el tratamiento técnico de los datos utilizados para el entrenamiento de la red neuronal.

Obtención de datos (dataset)

Para la obtención de los datos, he utilizado la plataforma **Roboflow Universe** [8]. Este es un repositorio internacional e interactivo especializado en visión artificial. Ofrece una inmensa variedad de conjuntos de datos públicos de múltiples ámbitos.

En este repositorio, se ha decidido utilizar un *dataset* específico compuesto por imágenes de bandejas sanitarias de colmenas con presencia etiquetada del ácaro *Varroa destructor*. Este conjunto de datos proporciona las foto-

grafías crudas junto a sus correspondientes anotaciones espaciales, lo cual es necesario para entrenar algoritmos de aprendizaje profundo en tareas de localización y detección de objetos.

Tratamiento del conjunto de datos

En cuanto al uso y preparación de los datos durante el proyecto, destacan principalmente los procesos aplicados antes de introducir las imágenes en la arquitectura de red neuronal:

Antes del entrenamiento (pre-entrenamiento)

Antes de proceder con el entrenamiento formal, es crucial asegurar la calidad y homogeneidad del conjunto de datos. Esto implica las siguientes etapas de diseño y adaptación:

- **Redimensionamiento y normalización:** Todas las imágenes se redimensionan a un tamaño espacial uniforme. Esto es para que se pueda garantizar que el modelo pueda agrupar y procesar los tensores de manera consistente y eficiente.
- **Aumentación de datos (*data augmentation*):** Para enriquecer el conjunto original y evitar el sobreajuste (*overfitting*), se aplican transformaciones sintéticas a las imágenes (rotaciones, cambios de brillo, recortes). Esto hace que el modelo sea mucho más robusto ante condiciones de captura del mundo real.
- **Formato de anotación:** Las etiquetas se han tratado utilizando el estándar de la industria Pascal VOC (mediante archivos XML), donde se detallan de manera precisa las coordenadas $(x_{min}, y_{min}, x_{max}, y_{max})$ de cada caja delimitadora alrededor de los parásitos.

Pesos preentrenados y transfer learning

En lugar de inicializar la red neuronal con valores aleatorios, utilizaremos un conjunto de pesos preentrenados correspondientes a la arquitectura base **ResNet-50** [3].

Esta estructura convolucional ya ha sido entrenada previamente utilizando el dataset global **MS COCO** [5], el cual es ampliamente utilizado para tareas de detección de objetos. Los pesos contenidos en este archivo corresponden a la configuración de la red tras haber aprendido a identificar

patrones visuales elementales (bordes, texturas, formas geométricas) en millones de imágenes.

Este archivo de pesos nos servirá como punto de partida, aprovechando el paradigma del aprendizaje transferido para adaptar el modelo preentrenado a nuestro nuevo conjunto de datos específico. Gracias a este diseño, evitamos tener que empezar el entrenamiento desde cero, reduciendo drásticamente el coste computacional y los tiempos de convergencia.

Después del entrenamiento (post-entrenamiento)

Una vez completados los ciclos (*epochs*) de entrenamiento del modelo Faster R-CNN [7], es indispensable evaluar su desempeño y analizar los datos resultantes antes de integrarlo en la aplicación web. Para ello, el tratamiento de los datos de salida se divide en dos fases analíticas:

1. **Evaluación de métricas de error y aprendizaje:** Se utiliza la biblioteca `matplotlib` para visualizar el comportamiento del modelo durante el proceso de entrenamiento. Esto incluye la representación gráfica de las curvas de la función de pérdida (*loss*) tanto para la clasificación como para la regresión de las cajas delimitadoras. Estas gráficas nos permiten identificar la evolución del aprendizaje, detectar de forma temprana problemas de sobreajuste o subajuste, y reajustar los hiperparámetros si la curva no converge adecuadamente.

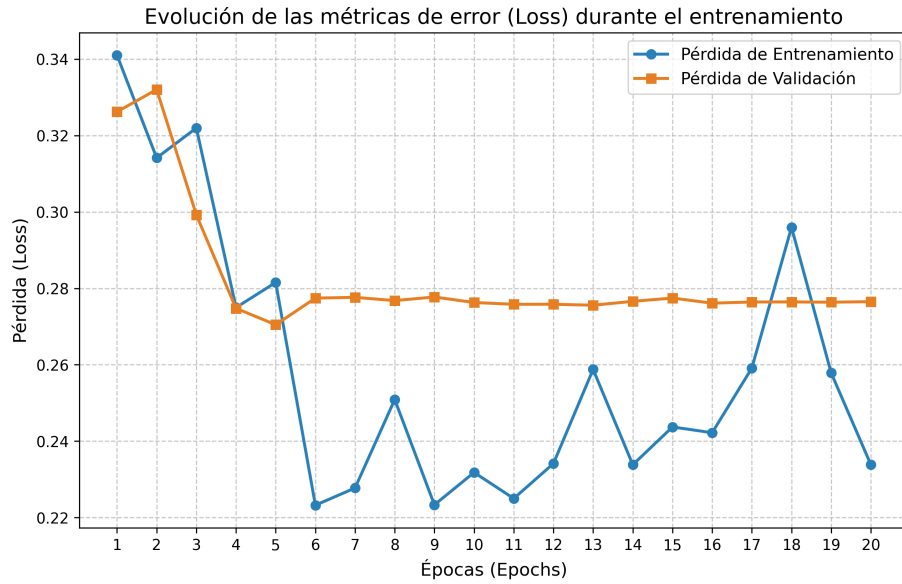


Figura C.1: Ejemplo de la evolución de las métricas de error durante el entrenamiento.

0.5

2. **Inferencia y generación de *bounding boxes*:** Una vez validado matemáticamente el rendimiento del modelo, se utilizan imágenes del conjunto de prueba (*test dataset*) que la red nunca ha visto para evaluar su eficacia. El modelo procesa los tensores de la imagen y genera las correspondientes bounding boxes alrededor de cada instancia detectada, indicando explícitamente que pertenece a la clase *Varroa destructor* junto con el porcentaje de precisión o nivel de confianza de dicha predicción.

- **role:** Tabla que define los niveles de privilegio del sistema (USER y ADMIN).
- **sesion_muestreo:** Actúa como cabecera del historial. Registra cada vez que un usuario realiza un análisis, guardando la fecha, el identificador del usuario creador, la cantidad total de fotos subidas en ese lote y la media matemática de varroas detectadas.
- **foto_detalle:** Tabla vinculada a la sesión de muestreo (relación de 1 a N). Almacena individualmente la ruta local de la imagen original subida, la ruta de la imagen procesada devuelta por la IA y el recuento exacto de parásitos encontrados en esa fotografía en concreto.

C.3. Diseño arquitectónico

El sistema implementa una arquitectura distribuida y desacoplada compuesta por dos bloques independientes que se comunican mediante servicios web bajo el paradigma REST. Esta separación de responsabilidades permite que el servidor web principal delegue la carga computacional de la inferencia a un microservicio especializado.

A continuación, se describen las tres capas fundamentales que integran el diseño del software:

Capa de presentación (frontend)

Esta capa representa la vista dentro de la arquitectura Modelo-Vista-Controlador del sistema. Las páginas web se construyen utilizando el motor de plantillas **Thymeleaf**, el cual ensambla dinámicamente el código HTML directamente en el servidor antes de enviarlo al navegador del usuario. Sus componentes principales son:

- **Estructura y diseño web:** La interfaz con la que interactúa el usuario se fundamenta en HTML5 y el *framework* **Bootstrap 5**. El uso de estas herramientas asegura una disposición adaptativa de los elementos visuales.
- **Visualización analítica:** La plantilla de historial `mis_analisis.html` integra la librería gráfica **Chart.js**. Este componente transforma automáticamente los vectores numéricos de infestación recuperados por el servidor en diagramas de líneas dinámicos, facilitando el seguimiento visual de la evolución de la plaga.

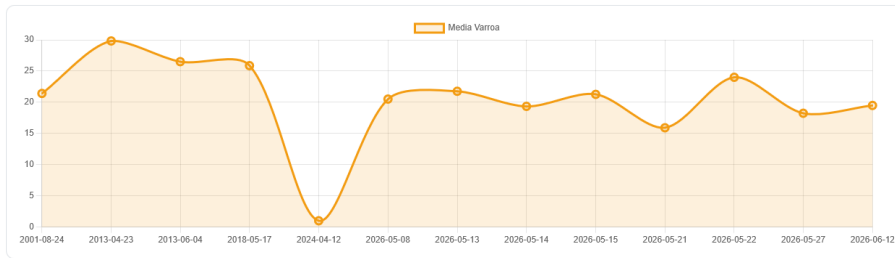


Figura C.3: Avance de la plaga a lo largo del tiempo.

1

- **Control de acceso en la interfaz:** La protección visual de las páginas web se gestiona mediante las etiquetas de Spring Security. Antes de enviar la web al navegador del usuario, el servidor comprueba sus permisos. Si un usuario estándar inicia sesión, el servidor elimina directamente el código HTML de los botones y paneles administrativos.

Servidor y procesamiento (backend)

El backend se divide en dos servicios coordinados: un orquestador principal (Spring Boot [10]) para la lógica de la aplicación y un microservicio (Flask) dedicado en exclusiva a la Inteligencia Artificial. Aquí se maneja la carga de archivos, la seguridad, la persistencia en base de datos y la inferencia del modelo. A continuación se muestra el desglose de sus responsabilidades:

Rutas y controladores (Spring Boot):

- **/, /login y /registro:** Renderizan las páginas de inicio, la pasarela de autenticación y la creación de nuevas cuentas de apicultor.
- **/nuevo-analisis:** Muestra el formulario para subir las imágenes de las colmenas, intercepta los archivos y los envía al servicio de IA.
- **/mis-analisis:** Consulta la base de datos y devuelve el historial de muestreos del usuario para generar las métricas.
- **/admin/usuarios:** Ruta protegida que permite a los administradores gestionar, listar y eliminar usuarios del sistema.

Procesamiento y detección de imágenes (Flask):

- **/predict**: Es la ruta de la API que recibe las imágenes, ejecuta el modelo de IA y devuelve el número de varroas encontradas.
- Se utiliza **OpenCV** (cv2) para decodificar las imágenes subidas, cambiar su espacio de color a RGB y dibujar los recuadros verdes sobre las varroas detectadas.
- Se emplean **Faster R-CNN** [7] y **PyTorch** [6], configurados para inferencia con los pesos del modelo preentrenado, para ejecutar la detección matemática de los parásitos.

Gestión de datos y seguridad (Spring Boot):

- Se utiliza **Spring Security** [10] para gestionar el inicio de sesión, encriptar las contraseñas de los apicultores y restringir el acceso a los paneles administrativos.
- Se hace uso de **Spring Data JPA** e **Hibernate** para persistir los datos de los usuarios y todo el historial de detecciones en una base de datos H2.
- Se dispone de una función de saneamiento activada manualmente mediante un botón en el panel de administración para optimizar el espacio de almacenamiento. Este mecanismo examina las carpetas físicas del servidor y elimina de forma definitiva aquellas imágenes huérfanas que no consten registradas en la base de datos.

Topología de red y puertos

Para garantizar el correcto enrutamiento en el entorno local (**localhost**) y evitar conflictos con otros servicios del sistema operativo, la arquitectura de despliegue se ha mapeado en los siguientes puertos específicos:

- **Puerto 8099 (Spring Boot [10])**: Es el puerto principal de entrada. El navegador del apicultor se conecta a este puerto para interactuar con la interfaz gráfica.
- **Puerto 5000 (Flask)**: Puerto local donde se levanta el microservicio de Inteligencia Artificial.

C.4. Diseño procedimental

En esta sección se detalla el flujo de ejecución y comunicativo del sistema, abarcando desde la interacción inicial del usuario en el frontend hasta el procesamiento en el backend y la posterior devolución de resultados. A continuación, se especifican los pasos secuenciales que conforman el ciclo de vida de una petición de análisis:

Flujo de detección y análisis de muestreos

1. **Petición del cliente:** El usuario accede a la vista de análisis y envía un formulario HTTP POST estructurado el cual contiene el lote de imágenes capturadas en el colmenar.
2. **Intercepción y preparación (Spring Boot):** El controlador principal recibe el flujo de archivos. Para cada imagen, el sistema genera un identificador universal unívoco (UUID) con el fin de prevenir colisiones en el sistema de archivos y almacena la imagen original físicamente en el servidor.
3. **Delegación de carga computacional:** El servidor Java suspende temporalmente el procesamiento de la vista y construye una nueva petición HTTP interna. Utilizando el componente cliente `RestTemplate`, reenvía la imagen al microservicio de visión artificial atacando el endpoint `/predict`.
4. **Inferencia neuronal (Flask):** El microservicio en Python recibe la petición y decodifica el archivo de imagen en formato binario, convirtiéndolo en un tensor normalizado para su procesamiento. A continuación, la red neuronal analiza la imagen para detectar la presencia de parásitos. Finalmente, se generan los cuadros delimitadores sobre las detecciones y la imagen resultante se guarda como una nueva versión procesada en disco.
5. **Respuesta del microservicio:** Flask finaliza su ejecución devolviendo a Spring Boot un objeto serializado en formato JSON (`AnalisisFotoDTO`), el cual contiene el recuento total de detecciones y la ruta de acceso a la nueva imagen marcada.
6. **Registro de datos:** El servidor principal recibe las métricas del microservicio y persiste el análisis de forma permanente en la base de datos H2 local.

7. **Redirección y respuesta al usuario:** Una vez asegurada la persistencia, Spring Boot ordena una redirección automática del navegador mediante una petición HTTP GET hacia la vista del historial. Thymeleaf renderiza los nuevos datos, presentando al apicultor las imágenes procesadas y la gráfica analítica actualizada.

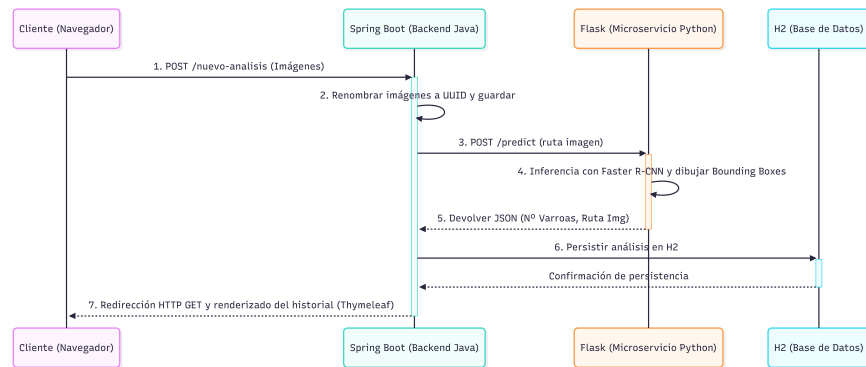


Figura C.4: Diagrama de secuencia de comunicación cliente-servidor para el proceso de análisis.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

Este apéndice proporciona la documentación técnica relativa al diseño estructural, organización de archivos y despliegue del sistema en el entorno de desarrollo.

D.2. Estructura de directorios

El espacio de trabajo del proyecto se organiza bajo el directorio raíz TFG VARROA. A continuación, se detalla la jerarquía del sistema de archivos:

- **/Modelo:** Alberga el entorno experimental de aprendizaje profundo. Contiene los conjuntos de datos originales (**DatasetOriginal**), los segmentos de distribución (**dataset_split**) y las rutinas de software en Python orientadas al procesamiento, ingeniería de datos y entrenamiento del modelo:
 - **Comprobar_Pares.py** y **Comprobar_BB.py**: Scripts encargados de verificar la integridad estructural del dataset mediante la correspondencia entre los archivos de imagen y sus respectivas anotaciones XML (*bounding boxes*).
 - **Aumentar_DataSet.py** y **División_dataset.py**: Scripts encargados de aplicar técnicas de aumento de datos mediante la biblioteca *Albumentations* [1] y realizar la partición del dataset en los

subconjuntos de entrenamiento (*train*), validación (*val*) y prueba (*test*).

- **Entrenamiento.py** e **Inferencia.py**: Módulos orientados al ciclo de optimización de pesos neuronales y a la ejecución de pruebas respectivamente.
 - **Evaluacion.py**: Script encargado de computar métricas de rendimiento del modelo, tales como la precisión media (*mAP*) y el índice de IoU.
- **/AppWeb**: Contiene toda la estructura del sistema que forma la página web.
- **/basico**: Módulo principal que engloba el backend y el frontend de la plataforma web, desarrollado bajo el framework Spring Boot [10]. Su estructura interna se divide en el código fuente de lógica y los recursos de la interfaz:
 - **/src/main/java**: Alberga los paquetes lógicos con las clases y lógica:
 - ◊ **config**: Clases encargadas de la configuración del sistema, incluyendo la seguridad y las conexiones a la base de datos local.
 - ◊ **controller**: Capa encargada de interceptar las peticiones HTTP del cliente, gestionar el flujo de navegación y coordinar las llamadas a los servicios.
 - ◊ **dto**: Objetos de Transferencia de Datos (*Data Transfer Objects*) utilizados para transportar información de forma segura entre la vista y el backend.
 - ◊ **model**: Clases de entidad que mapean el modelo de datos del sistema (estructuras de usuarios, roles y muestreos) mediante JPA/Hibernate.
 - ◊ **repository**: Interfaces encargadas de la abstracción y acceso a los datos persistentes mediante la ejecución de consultas sobre la base de datos H2 [2].
 - **/src/main/resources**: Directorio que almacena los recursos de configuración y la capa de presentación (frontend):
 - ◊ **application.properties**: Archivo central de configuración donde se definen los parámetros de la base de datos local H2 [2], el puerto de despliegue y los tamaños máximos de los archivos cargados.

- ◊ **/templates:** Carpeta donde se almacenan de forma exclusiva todas las vistas e interfaces dinámicas en formato de plantillas HTML (*Thymeleaf*).
- **/api-python:** Microservicio implementado en Flask integrado por el script de inferencia `app.py` y el directorio `/static`, el cual divide las imágenes en `/uploads` para registrar las muestras de entrada y `/results` para almacenar las capturas procesadas con las detecciones de varroa.

D.3. Manual del programador

Este apartado describe los componentes físicos necesarios para el funcionamiento del sistema, así como las herramientas utilizadas durante el desarrollo del software.

Requisitos de hardware

Para que la aplicación funcione de forma correcta y procese las imágenes de las colmenas con fluidez, se recomiendan las siguientes especificaciones físicas:

- **Procesador:** Cualquier procesador con múltiples núcleos y capacidad de cómputo suficiente (Intel Core i5, AMD Ryzen 5 o superior).
- **Memoria RAM:** Mínimo 8 GB.
- **Tarjeta gráfica:** Se recomienda una tarjeta gráfica NVIDIA con soporte CUDA para acelerar el procesamiento de las imágenes; si no se dispone de ella, el sistema utilizará el procesador del ordenador automáticamente.

Requisitos de software y entorno de desarrollo

Todo el código del proyecto se ha programado y organizado utilizando el editor de texto *Visual Studio Code*. En la Tabla [D.1](#) se detallan las versiones de los programas y herramientas principales que dan vida al sistema:

Tabla D.1: Herramientas y versiones de software utilizadas.

Programa / Framework	Versión mínima
Java (JDK)	17.0 o superior
Spring Boot (Framework Web) [10]	3.4.x
Python (Intérprete)	3.9 o superior
PyTorch (Librería de IA) [6]	2.0 o superior
OpenCV (Librería de Imagen)	4.0 o superior
H2 Database (Base de datos) [2]	Embebida (2.x)

D.4. Compilación, instalación y ejecución del proyecto

Para facilitar la puesta en marcha de la aplicación sin necesidad de realizar configuraciones manuales en el sistema, todo el proceso se ha automatizado mediante el script `iniciar_todo.bat` situado en la raíz del proyecto.

Descarga del proyecto (clonado del repositorio)

Para obtener una copia local de todo el código fuente, los conjuntos de datos y los pesos del modelo de Inteligencia Artificial, es necesario utilizar **Git**.

Abre una terminal en la carpeta donde desees descargar el proyecto y ejecuta el siguiente comando:

```
git clone https://github.com/ManuelMelendezOrtega/TFG-Deteccion-varroa.git
```

Requisitos previos

Antes de ejecutar la aplicación en un ordenador nuevo, es necesario comprobar que se encuentran instalados los siguientes programas base:

- **Git:** Necesario para poder descargar y clonar correctamente el repositorio del proyecto desde GitHub.
- **Java Development Kit (JDK) 17 o superior:** Necesario para poder arrancar el servidor web principal.
- **Python (Versión 3.9 o superior):** Necesario para poder ejecutar el microservicio de inteligencia artificial.

Instalación y ejecución

Una vez instalados los programas anteriores, los pasos para arrancar el sistema completo se reducen a:

1. Situar en la raíz de la carpeta `TFG VARROA` o `TFG-Deteccion-varroa` si se ha clonado el repositorio.
2. Hacer doble clic sobre el archivo `iniciar_todo.bat`.
3. El script se encargará automáticamente de inicializar el entorno local: creará el entorno virtual de Python (`.venv`), instalará las librerías de IA necesarias en su primera ejecución y arrancará el servidor Java junto a la base de datos embebida H2 [2].
4. Cuando el script finalice, la aplicación estará lista para usarse abriendo cualquier navegador web e introduciendo la dirección: `http://localhost:8099`, se sabrá que tanto Spring Boot como el microservicio en Python están listos cuando aparezcan los siguientes textos en las consolas que aparecerán al ejecutar el archivo.

```
2026-05-29T10:29:06.328+02:00 INFO 29580 --- [basico] [main] c.s.basico.BasicoAppl
ication : Started BasicoApplication in 4.161 seconds (process running for 4.668)
```

Figura D.1: Consola de comandos para Spring Boot lista.

1

```
>>> [OK] IA lista para trabajar.
* Debugger is active!
* Debugger PIN: 560-798-544
```

Figura D.2: Consola de comandos para el microservicio lista.

1

Activación del entorno virtual (ejecución manual de scripts)

Si se desea ejecutar cualquiera de los scripts de Python de forma manual (como los de entrenamiento `Entrenamiento.py`, evaluación `Evaluacion.py` o pruebas de inferencia), es necesario activar primero el entorno virtual. Esto garantiza que se utilicen las versiones exactas de las librerías (PyTorch, OpenCV, etc.) requeridas por el proyecto y empaquetadas en la carpeta `.venv`.

Desde la raíz del proyecto, ejecute el siguiente comando según su sistema operativo:

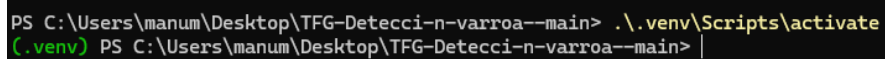
- **En Windows (Command Prompt o PowerShell):**

```
.\.venv\Scripts\activate
```

- **En Linux / macOS:**

```
source .venv/bin/activate
```

(Nota: Se Sabrá que el entorno se ha activado correctamente porque aparecerá el prefijo `(.venv)` al principio de la línea de la terminal. Para salir del entorno virtual una vez se haya terminado de probar los scripts, simplemente se ejecutará el comando `deactivate`).



```
PS C:\Users\manum\Desktop\TFG-Detecci-n-varroa--main> .\.venv\Scripts\activate  
(.venv) PS C:\Users\manum\Desktop\TFG-Detecci-n-varroa--main> |
```

Figura D.3: Línea de comandos con el entorno activo.

D.5. Pruebas del sistema

Durante el desarrollo del proyecto, se han llevado a cabo diferentes metodologías de prueba para asegurar que tanto el entorno de inteligencia artificial como la aplicación web funcionan correctamente, tanto por separado como en conjunto.

Pruebas de los datos y del modelo

El entrenamiento de una red neuronal requiere que los datos sean perfectos. Para validar esta fase se han implementado las siguientes comprobaciones:

- **Integridad del dataset:** Se desarrollaron los scripts `Comprobar_Pares.py` y `Comprobar_BB.py`. Su objetivo es actuar como un filtro de calidad que recorre los archivos para asegurar que a ninguna fotografía le falta su archivo de etiquetas XML y que las cajas delimitadoras (*bouding boxes*) se coloquen correctamente en las imágenes nuevas tras el aumento del dataset.
- **Control de sobreajuste:** Durante el entrenamiento, el sistema evalúa automáticamente el aprendizaje de la IA generando gráficas que comparan el error de entrenamiento (*train_loss*) con el error de validación (*val_loss*). Estas gráficas sirven como prueba visual para comprobar que el modelo realmente está aprendiendo a detectar varroas nuevas y no limitándose a memorizar las fotos que ya ha visto.

Pruebas de la aplicación web

Para comprobar la solidez del servidor web, se ha hecho uso de las herramientas de prueba que el framework *Spring Boot* [10] trae integradas de base (mediante la librería *JUnit*):

- **Pruebas de contexto y conexión:** Antes de compilar el código final, Spring Boot realiza pruebas automáticas que simulan el arranque de la aplicación. Esto sirve para certificar que no hay errores de código graves, que las credenciales de los archivos de configuración son correctas y que la conexión con la base de datos se realiza sin que el programa se cierre inesperadamente.

Pruebas de depuración en Visual Studio Code

- **Uso del Debugger:** Se utilizó la herramienta de depuración integrada en *Visual Studio Code*. Esta herramienta permitió pausar la ejecución del código línea por línea en ambos lenguajes a la vez. De esta forma, se pudo seguir el rastro completo de una foto: desde que el usuario hace clic en la web, pasando por su conversión a datos numéricos,

hasta comprobar que el servicio de Python en el puerto 5000 devolvía correctamente las coordenadas de los parásitos detectados.

Apéndice *E*

Documentación de usuario

E.1. Introducción

Este apéndice funciona como la guía de usuario de la plataforma web. Su objetivo principal es ofrecer un manual claro, directo y muy visual para que cualquier usuario, sin importar su nivel de conocimientos informáticos, sea capaz de poner en marcha la aplicación en su ordenador, registrarse y empezar a analizar las imágenes de sus colmenas.

E.2. Requisitos de usuarios

Para que un apicultor o usuario final pueda utilizar el sistema en su propio ordenador con total fluidez, es necesario que su equipo cumpla con una serie de requisitos básicos de hardware y software:

- **Sistema operativo:** Ordenador con Microsoft Windows 10 o Windows 11 (versiones de 64 bits).
- **Hardware recomendado:** Aunque la aplicación se adapta a equipos modestos, se aconseja contar con un procesador Intel Core i5 o AMD Ryzen 5 (o equivalentes) y un mínimo de 8 GB de memoria RAM para que el análisis de las imágenes sea rápido.
- **Programas necesarios (preinstalación):** Al ser una aplicación que se ejecuta de forma local en el ordenador del usuario, este debe instalar previamente dos herramientas:

- *Java (JDK 17 o superior)*: El motor que permite arrancar y hacer funcionar la página web principal y la base de datos embebida.
 - *Python (3.9 o superior)*: El entorno que ejecuta el modelo de inteligencia artificial. Es importante activar la opción *Add Python to PATH* durante su instalación para que el sistema lo encuentre.
- **Conexión a internet:** Solo es obligatoria la primera vez que se vaya a usar el programa para permitir que se descarguen las librerías necesarias. Una vez hecha esta primera configuración, el programa puede usarse en un entorno completamente desconectado (por ejemplo, en el campo).
 - **Navegador de internet:** Cualquier navegador moderno y actualizado como *Google Chrome*, *Microsoft Edge*, *Mozilla Firefox* o *Brave*.

E.3. Instalación

Gracias al script automatizado que incluye el proyecto, el proceso de instalación y arranque se ha simplificado para que pueda realizarse en un par de clics:

1. **Lanzar la aplicación:** El usuario debe entrar en la carpeta raíz del proyecto (TFG VARROA) y hacer doble clic sobre el archivo llamado `iniciar_todo.bat`.
2. **Espera de configuración:** Al ejecutar el archivo se abrirán varias ventanas de comandos de Windows. El usuario no debe tocar nada ni cerrar ninguna ventana; el asistente se encargará de inicializar la base de datos H2 [2], activar el entorno de inteligencia artificial y encender el servidor web de manera transparente.
3. **Acceso a la plataforma:** Cuando las ventanas terminen de cargar y muestren los mensajes de confirmación, la aplicación estará lista. El usuario solo tendrá que abrir su navegador habitual e introducir en la barra de direcciones la siguiente ruta local: `http://localhost:8099`.

Al introducir dicha dirección, la plataforma responderá cargando la pantalla de bienvenida y el formulario de acceso que se detalla en el siguiente apartado del manual.

E.4. Manual del usuario

Una vez que el usuario accede a la dirección local del servidor (`http://localhost:8099`), se le presentará directamente la pantalla de inicio de sesión.

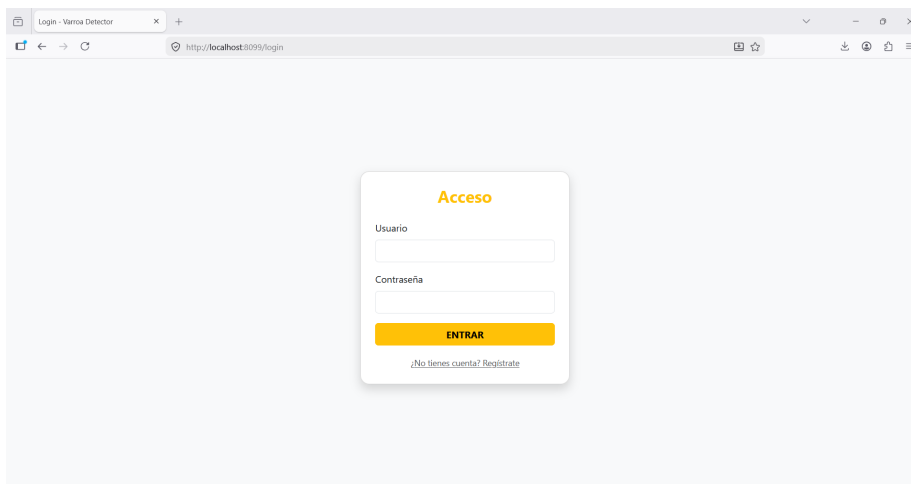


Figura E.1: Pantalla de login.

1

Si todavía no se tiene un usuario registrado, se puede crear una cuenta nueva accediendo al formulario a través del botón “¿No tienes cuenta? Regístrate”, ubicado justo debajo del botón de “ENTRAR”.

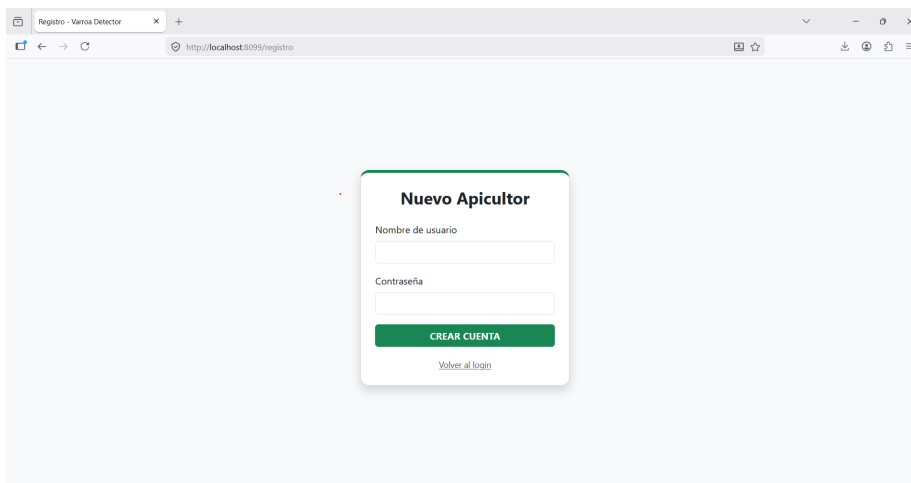


Figura E.2: Pantalla de registro.

1

Al acceder correctamente con las credenciales, aparecerá la pantalla principal de la aplicación. En ella disponemos de tres opciones claras: el botón **“NUEVO ANÁLISIS”** para subir imágenes, la opción **“VER MI HISTORIAL”** para consultar diagnósticos pasados, y el botón de **“Cerrar Sesión”** situado en la parte superior para salir de la cuenta.

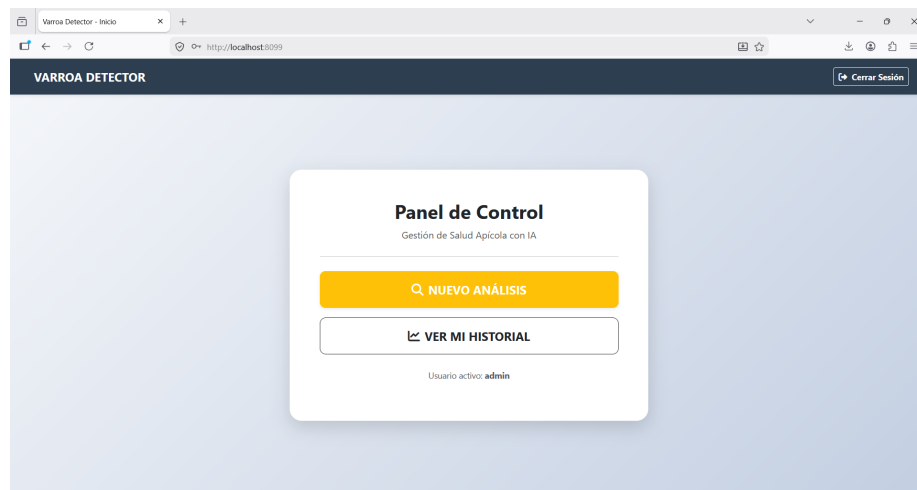


Figura E.3: Pantalla principal.

1

En la página de nuevo análisis, el apicultor puede seleccionar la fecha exacta en la que se recogió la muestra y cargar las imágenes que desea procesar, permitiendo un máximo de 100 archivos por lote. Al presionar el botón **“Procesar con IA”**, comenzará la inferencia del modelo. Este proceso tarda aproximadamente un segundo por fotografía (aunque varía según el hardware del equipo) y, al finalizar, redirige automáticamente al historial. También se incluye un botón de “Cancelar” para regresar al menú anterior sin guardar cambios.

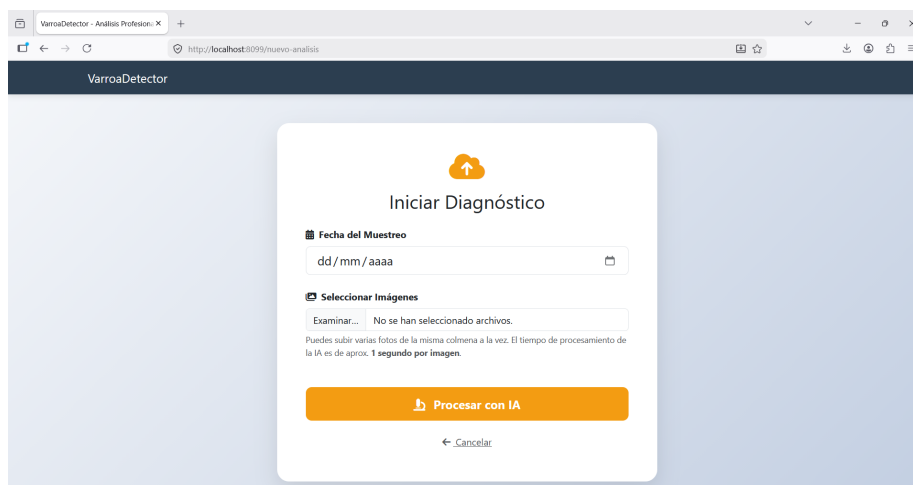


Figura E.4: Pantalla para nuevos análisis.

1

Dentro de la pantalla del historial se despliega una gráfica interactiva de líneas. Cada punto de la gráfica representa la media de varroas detectadas en los lotes analizados ese día. En la parte superior de este panel se muestran tres métricas: la media global de parásitos, el valor máximo detectado y el número total de análisis computados. Estos datos se actualizan dinámicamente, ya que el usuario puede filtrar el rango temporal del que se quieren usar análisis para la gráfica y se puede visualizar de manera diaria, mensual o anual mediante los selectores así como restablecerlos con el botón de reinicio.

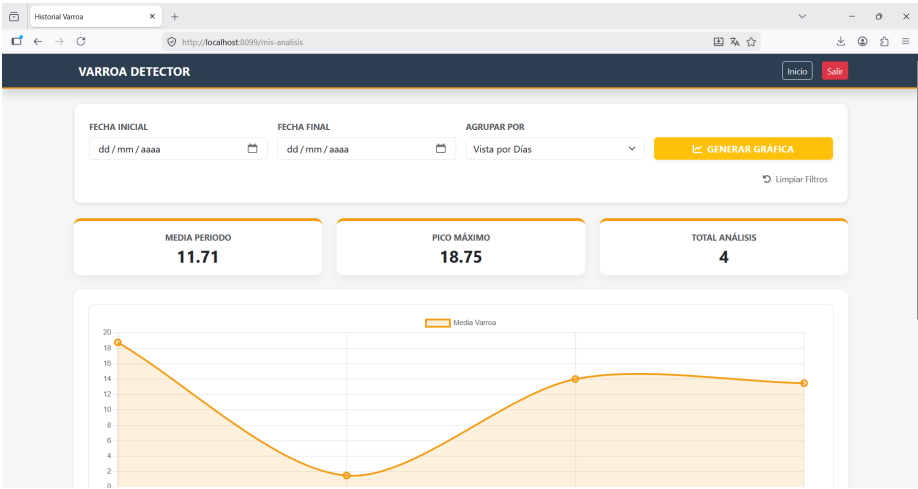


Figura E.5: Parte superior de la pantalla del historial.

1

En la parte inferior de esta misma pantalla aparece una tabla detallada con cada uno de los lotes realizados. Si se ha aplicado un filtro temporal previo, la tabla se adaptará para mostrar solo los registros de esas fechas. Por cada lote se especifica el número de imágenes procesadas y la media de varroas calculada, ofreciendo un botón para eliminar un análisis y el botón “Ver” para inspeccionar los resultados.

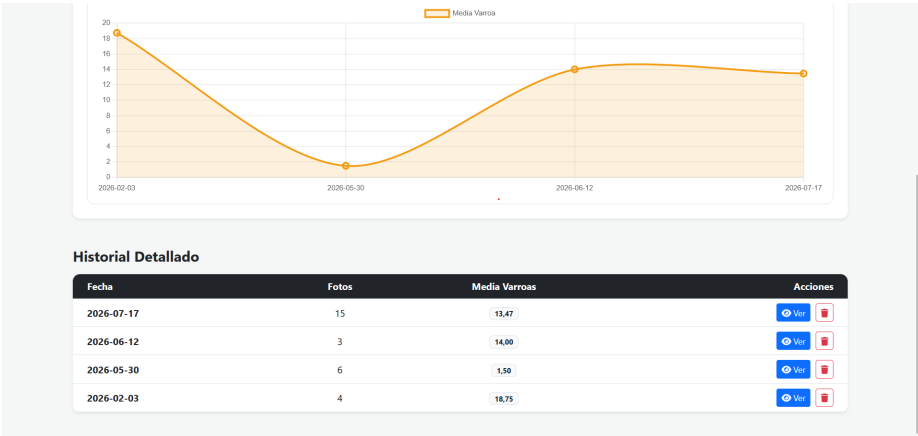


Figura E.6: Parte inferior de la pantalla del historial.

1

Al pulsar en “Ver”, la plataforma presenta las imágenes del lote. Aquí se examina cada fotografía de forma individual, detallando el conteo de ácaros por imagen y con los recuadros dibujados sobre cada parásito. Además, se añade el botón “Descargar” en cada archivo para permitir al apicultor almacenar de forma local la imagen marcada si requiere un estudio más riguroso.

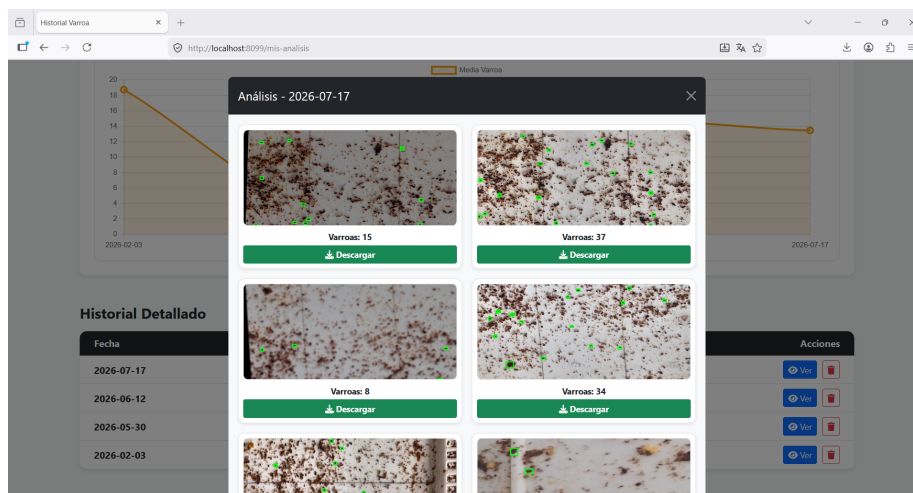


Figura E.7: Opción para visualizar las imágenes con las detecciones de la IA.

1

Finalmente, si el usuario que inicia sesión posee el rol de administrador, la barra de navegación superior renderizará el botón “Panel Admin” al lado de la opción de cerrar sesión.

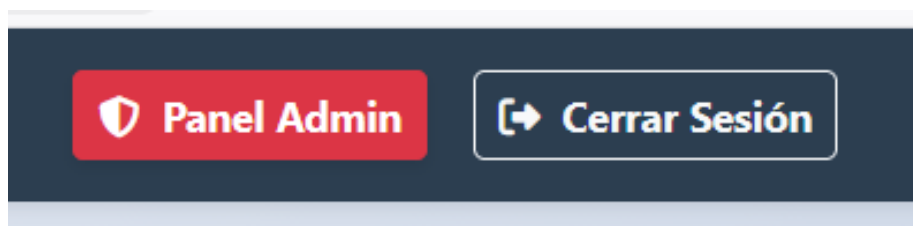


Figura E.8: Botón para acceder al panel admin.

1

Dentro del panel de administración se despliega el listado completo de los usuarios registrados. Desde aquí se puede dar de baja a cualquier

usuario eliminando físicamente toda su información de la base de datos y sus imágenes. Asimismo, el administrador dispone del botón de saneamiento para purgar de las carpetas de almacenamiento todas aquellas imágenes huérfanas que no estén ligadas a ningún análisis.

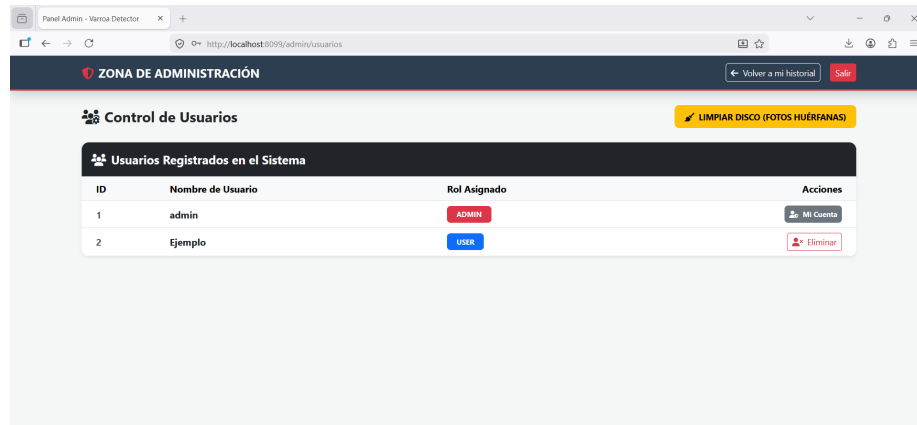


Figura E.9: Pantalla de administración.

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo reflexiona sobre la integración de los principios de sostenibilidad en el Trabajo de Fin de Grado. Mediante la aplicación de inteligencia artificial para detectar el ácaro *Varroa destructor*, se demuestra cómo las competencias en Ingeniería Informática pueden alinearse con el cuidado medioambiental y la viabilidad económica, tomando como referencia los Objetivos de Desarrollo Sostenible de la Agenda 2030.

F.2. Conocimiento y conciencia ambiental

Durante el proyecto se investigó la situación crítica de las abejas y su impacto en la polinización mundial. El ácaro *Varroa destructor* es una de sus mayores amenazas. Históricamente, esta plaga se ha combatido aplicando tratamientos químicos de forma preventiva o mediante estimaciones imprecisas. La solución desarrollada fomenta una apicultura de precisión. Al cuantificar el grado de infestación, los apicultores pueden aplicar acaricidas estrictamente cuando se supera el umbral de peligro. Esta reducción de pesticidas minimiza la toxicidad residual, protegiendo la biodiversidad del ecosistema y la calidad de la miel y la cera.

F.3. Innovación y tecnología para la sostenibilidad

Modernizar el sector rural es vital. El proyecto impulsa la innovación tecnológica implementando redes neuronales convolucionales, específicamente Faster R-CNN, adaptadas al reconocimiento de parásitos. Esta tecnología proporciona un diagnóstico rápido y automatizado que sustituye al conteo manual, propenso a errores. Facilitar decisiones basadas en datos precisos promueve prácticas agrícolas sostenibles y una gestión proactiva de la salud animal.

F.4. Despliegue local y accesibilidad tecnológica

A diferencia de soluciones que dependen de infraestructuras centralizadas, este sistema utiliza el procesamiento local. Usar Spring Boot y la base de datos embebida H2 permite que la aplicación funcione de forma totalmente autónoma y *offline*. Esta decisión arquitectónica es esencial para el entorno rural, donde la conectividad a Internet suele ser limitada, garantizando que la tecnología sea operativa directamente en los colmenares.

F.5. Responsabilidad social y ética profesional

Distribuir el software bajo la Licencia MIT evidencia un claro compromiso social. Esta elección elimina barreras económicas, permitiendo que apicultores de cualquier nivel adquisitivo puedan utilizar y auditar el sistema libremente. Se combate así la brecha digital en el sector primario, democratizando el acceso a herramientas avanzadas y fomentando la colaboración tecnológica frente a un problema ecológico global.

F.6. Aplicación de competencias

Desarrollo sostenible de software

El ciclo de vida del proyecto empleó metodologías ágiles integradas con CRISP-DM. Esta combinación aseguró una utilización eficiente de los

recursos, reduciendo el desperdicio de horas de programación y alineándose con la máxima eficiencia en la ingeniería de software.

Optimización y eficiencia energética

Frente a las aplicaciones en la nube que requieren servidores encendidos ininterrumpidamente, el diseño local promueve el *Green Computing*. Al prescindir de centros de datos externos y de la transmisión masiva de imágenes por red, se elimina la huella de carbono asociada. El consumo energético se limita al hardware del usuario y solo durante el tiempo de análisis.

Reducción del uso de recursos computacionales

El entrenamiento de arquitecturas de aprendizaje profundo exige un alto coste energético. Para mitigarlo, se aplicó *Transfer Learning* utilizando un modelo preentrenado con el conjunto MS COCO mediante PyTorch. Al elegir este modelo preentrenado y limitar el ajuste a únicamente 20 épocas utilizando una gráfica local NVIDIA RTX 4060 Ti, el consumo energético derivado del entrenamiento ha sido drásticamente inferior al que habría supuesto entrenar una red neuronal profunda desde cero. Esta estrategia alinea directamente el proyecto con las prácticas de *Green Computing* y la Inteligencia Artificial sostenible. Además, este ahorro energético y bajo coste computacional demuestran que el modelo es altamente eficiente, haciéndolo viable para su futura implementación en entornos de *Edge Computing*, lo que permitiría procesar las imágenes directamente en dispositivos IoT ligeros en el propio colmenar.

F.7. Reflexión personal

Este trabajo ha transformado mi visión sobre el rol del ingeniero informático. Tradicionalmente, nos centramos en la complejidad algorítmica, ignorando el impacto ambiental. Sin embargo, aplicar la inteligencia artificial para proteger a las abejas demuestra empíricamente que nuestras decisiones de diseño repercuten en el mundo físico.

Desarrollar una herramienta técnicamente solvente que protege la biodiversidad, apoya al entorno rural y minimiza el derroche computacional me ha proporcionado una perspectiva integral. Estas competencias en sostenibilidad constituyen un pilar ético fundamental para mi futuro profesional.

Reafirmo mi compromiso de diseñar tecnología que actúe como un motor de cambio equitativo y respetuoso con el medio ambiente.

Bibliografía

- [1] Alexander Buslaev, Vladimir I Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- [2] H2 Group. H2 database engine, 2024. Último acceso el 17 de Mayo de 2026.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [4] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [5] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [6] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
- [7] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 28, 2015.

- [8] Roboflow, Inc. Roboflow universe: Open source computer vision datasets and apis, 2024. Último acceso el 17 de Mayo de 2026.
- [9] Peter Rosenkranz, Pia Aumeier, and Bettina Ziegelmann. Biology and control of varroa destructor. *Journal of Invertebrate Pathology*, 103:S96–S119, 2010.
- [10] VMware, Inc. Spring boot official documentation, 2024. Último acceso el 17 de Mayo de 2026.
- [11] Rüdiger Wirth and Jochen Hipp. Crisp-dm: Towards a standard process model for data mining. In *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, pages 29–39. Springer-Verlag, 2000.